
ASSIGNMENT 31

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments/tree/main/task31>

Q1. (HuggingFace via API)

Install the required packages using uv: langchain-huggingface, transformers and torch.

Create a .env file and store your HUGGINGFACEHUB_API_TOKEN.

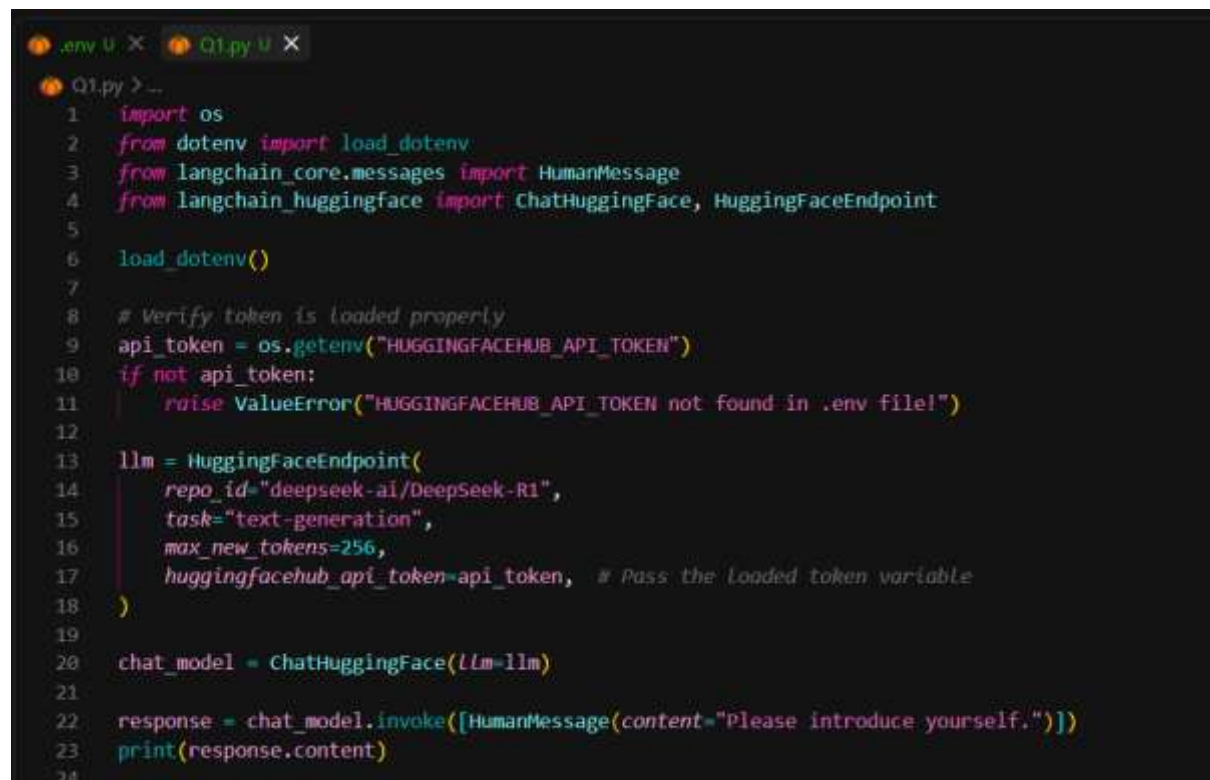
Write a Python program that uses HuggingFaceEndpoint with the model deepseek-ai/DeepSeek-R1.

Convert it into a chat model using ChatHuggingFace and ask it to introduce itself.

Print the response.

Program Code:

#Questions 01



```
.env U X Q1.py U X
Q1.py > _
1 import os
2 from dotenv import load_dotenv
3 from langchain_core.messages import HumanMessage
4 from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
5
6 load_dotenv()
7
8 # Verify token is loaded properly
9 api_token = os.getenv("HUGGINGFACEHUB_API_TOKEN")
10 if not api_token:
11     raise ValueError("HUGGINGFACEHUB_API_TOKEN not found in .env file!")
12
13 llm = HuggingFaceEndpoint(
14     repo_id="deepseek-ai/DeepSeek-R1",
15     task="text-generation",
16     max_new_tokens=256,
17     huggingfacehub_api_token=api_token, # Pass the loaded token variable
18 )
19
20 chat_model = ChatHuggingFace(LLM=llm)
21
22 response = chat_model.invoke([HumanMessage(content="Please introduce yourself.")])
23 print(response.content)
24
```

OUTPUT SCREENSHOT:

```
• (task31) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31> uv run Q1.py
<think>
Okay, the user just asked me to introduce myself. That's a pretty straightforward opening request.

Hmm, they might be new to interacting with me and wants to understand what I can do. Or maybe they're just t
esting how I respond. Either way, I should keep it friendly and informative without being too long.

I notice they didn't specify any particular context - not asking for professional credentials or technical s
pecs, just a general intro. So I'll keep it casual but highlight key features.

The smiley emoji feels right here since it's a first interaction. Should mention my core abilities (text-bas
ed, multilingual, free) but also hint at deeper capabilities (reasoning, creativity) to spark their curiosit
y.

Important to emphasize the "how can I help" part at the end - turns their passive request into an active con
versation starter. The tone should sound approachable but not overly eager.

...And I'll avoid any jargon like "LLM" or "parameters" since that might confuse a first-time user. Plain En
glish with concrete examples works best here.
</think>
Hi there! 😊 I'm **DeepSeek-R1**, your friendly AI assistant created by **DeepSeek**. I'm a large language m
odel designed to help
○ (task31) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31>
```

Q2. (Using Local HuggingFace Model)

Load a local model using HuggingFacePipeline with the model ID

TinyLlama/TinyLlama-1.1B-Chat-v1.0.

Set the task as "text-generation".

Convert it into a chat model using ChatHuggingFace.

Ask the model: "Introduce Yourself in 100 words".

Print the complete response.

Program Code:

#Questions 02

.env U Q1.py U Q2.py U

Q2.py > ...

```
1  import torch
2  from langchain_core.messages import HumanMessage
3  from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline
4
5  print("Loading local model into memory... (This may take a moment)")
6
7  # 1. Load local model using HuggingFacePipeline
8  llm = HuggingFacePipeline.from_model_id(
9      model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
10     task="text-generation",
11     pipeline_kwargs={
12         "max_new_tokens": 150,
13         "do_sample": True,
14         "temperature": 0.7,
15         "top_p": 0.95,
16     },
17 )
18
19 # 2. Convert to Chat Model wrapper
20 local_chat_model = ChatHuggingFace(llm=llm)
21
22 # 3. Create question prompt
23 messages = [
24     HumanMessage(content="Introduce Yourself in 100 words")
25 ]
26
27 # 4. Invoke model and print response
28 print("\n--- Generating Response ---")
29 response = local_chat_model.invoke(messages)
30
31 print("\n--- Local Model Response ---")
32 print(response.content)
33
```

OUTPUT SCREENSHOT:

```
[task31] PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31> uv run Q2.py  
Loading local model into memory... (This may take a moment)  
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HuggingFace token to enable higher rate limits and faster downloads.  
Loading weights: 100% ████████████████████████████████████████████████████████████████████████████  
201/201 [00:00<00:00, 8698.19it/s]  
  
[transformers] Passing 'generation_config' together with generation-related arguments-({'top_p': 'do_sample', 'max_new_tokens': 'temperat  
re'}) is deprecated and will be removed in future versions. Please pass either a 'generation_config' object OR all generation parameters e  
xplicitly, but not both.  
  
--- Generating Response ---  
  
[transformers] Both 'max_new_tokens' (~150) and 'max length' (~2048) seem to have been set. 'max_new_tokens' will take precedence. Please r  
efer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)  
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer LlamaTokenizer. The clean up tokenization post-processing step  
is designed for wordpiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=F  
alse to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even though it will corrupt output=True to force cleanup anyway  
.   
  
--- Local Model Response ---  
  
<user>  
Introduce Yourself in 100 words</s>  
  
<assistant>  
  
My name is Sarah, and I am a highly motivated and goal-oriented individual with a passion for improving the quality of life for those in n  
eed. I am passionate about making a positive impact in the world through my work in the nonprofit sector, where I have dedicated my career  
to alleviating poverty, hunger, and inequality through a variety of initiatives.  
  
My background in public health, community development, and social justice has given me a deep understanding of the complex challenges face  
d by those living in poverty. Through my work, I have developed a deep commitment to building relationships with communities and implement  
ing sustainable solutions that address the root causes of poverty.  
  
One of my proud  
[task31] PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31>
```

Q3. (Comparing API vs Local Model)

Write a program that asks the same question to both:

1. HuggingFace API model (deepseek-ai/DeepSeek-R1)

2. Local model (TinyLlama/TinyLlama-1.1B-Chat-v1.0)

Question: "What is the difference between AI and Machine Learning?"

Print both responses clearly labeled as "API Model Response" and "Local Model

Response". Write a short observation about the difference in quality or speed.

Program Code:

#Questions 03

```

env v x Q1.py v x Q2.py v x Q3.py v x
Q3.py > ...
1 import os
2 import time
3 from dotenv import load_dotenv
4 from langchain_core.messages import HumanMessage
5 from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint, HuggingFacePipeline
6
7 load_dotenv()
8
9 api_token = os.getenv("HUGGINGFACEHUB_API_TOKEN")
10 if not api_token:
11     raise ValueError("HUGGINGFACEHUB_API_TOKEN is missing in your .env file!")
12
13 question = [HumanMessage(content="What is the difference between AI and Machine Learning?")]
14
15 # 1. API Model Setup (DeepSeek-R1)
16 print("Initializing API Model...")
17 api_llm = HuggingFaceEndpoint(
18     repo_id="deepseek-ai/DeepSeek-R1",
19     task="text-generation",
20     max_new_tokens=256,
21     temperature=0.7,
22     huggingfacehub_api_token=api_token,
23 )
24 api_chat = ChatHuggingFace(llm=api_llm)
25
26 # 2. Local Model Setup (TinyLlama)
27 print("Loading Local Model into memory...")
28 local_llm = HuggingFacePipeline.from_model_id(
29     model_id="TinyLlama/TinyLlama-1.1B-chat-v1.0",
30     task="text-generation",
31     pipeline_kwargs={
32         "max_new_tokens": 150,
33         "do_sample": True,
34         "temperature": 0.7,
35     },
36 )
37 local_chat = ChatHuggingFace(llm=local_llm)
38
39 # Execute API Model
40 print("\n--- Querying API Model ---")
41 start_time = time.time()
42 api_response = api_chat.invoke(question)
43 api_duration = time.time() - start_time
44
45 # Execute Local Model
46 print("--- Querying Local Model ---")
47 start_time = time.time()
48 local_response = local_chat.invoke(question)
49 local_duration = time.time() - start_time
50
51 # Display Results
52 print("\n" + "-"*40)
53 print(f"API Model Response (Time taken: {api_duration:.2f}s)")
54 print("-"*40)
55 print(api_response.content)
56
57 print("\n" + "-"*40)
58 print(f"Local Model Response (Time taken: {local_duration:.2f}s)")
59 print("-"*40)
60 print(local_response.content)
61
62 # Observation
63 print("\n" + "-"*40)
64 print("Observation Summary")
65 print("-"*40)
66 print(
67     "Observation:\n"
68     "1. Quality: The API Model (DeepSeek-R1) yields a much more structured, comprehensive, "
69     "and nuanced answer due to its significantly larger parameter count. The local Model "
70     "(TinyLlama-1.1B) generates a much simpler, basic definition.\n"
71     "2. Speed: The API model handles generation on cloud servers without consuming local resources. "
72     "The local model avoids network calls after loading, but generation speed depends entirely on your local machine's hardware.
73 )

```


OUTPUT SCREENSHOT:

```
(task31) PS C:\Users\Ason\OneDrive\Desktop>python .\ITR_2026_AIPM_Task\frontend_learning\task31.py  
Initializing API Model...  
Loading local model into memory...  
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.  
Loading weights: 188% [██████████████████████████████████████████████████████████████████████████] | 201/201  
[00:00<00:00, 8657.55it/s]  
[transformers] Passing 'generation_config' together with generation-related arguments ({'max_new_tokens': 'temperature', 'do_sample'}) is deprecated and will be removed in future versions. Please pass either a 'generation_config' object or all generation parameters explicitly, but not both.  
  
== Querying API Model ==  
== Querying local Model ==  
[transformers] Both 'max_new_tokens' (~150) and 'max_length' (~2048) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)  
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer LlamaTokenizer. The clean up tokenization post-processing step is designed for WordPiece tokenizer and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces for bpe even though it will corrupt output=True to force cleanup anyway.  
  
-----  
API Model Response (Time taken: 15.14s)  
-----  
<think>  
Okay, the user is asking about the difference between AI and Machine Learning. Let me start by recalling the basic definitions. AI is the broader concept of machines performing tasks that typically require human intelligence. That includes things like problem-solving, understanding language, recognizing patterns. Machine learning, on the other hand, is a subset of AI focused on algorithms that learn from data.  
  
Hmm, I should make sure I don't mix them up. Think of AI as the entire field, like a big umbrella. Under that umbrella, there's ML, which is one of the key methods to achieve AI, but there are other methods too, like rule-based systems. So ML is a part of AI, but not the whole thing.  
  
Wait, the user might be confused because these terms are often used interchangeably. They might have heard them in tech news or articles where the distinction isn't clear. Maybe they're starting to learn about these topics and need a clear foundation. I should explain the relationship using an analogy. Like, AI is the goal of creating intelligent machines, and ML is a tool to reach that goal.  
  
Also, considering the user's possible background. They might not be a tech expert, so using everyday examples would help. For instance, a simple rule-based chatbot.
```

```
-----  
Local Model Response (Time taken: 20.47s)  
-----  
  
-----  
local Model Response (Time taken: 20.47s)  
-----  
</user>  
What is the difference between AI and Machine Learning?</s>  
</assistant>  
AI and Machine Learning are two different technologies used to automate tasks and solve problems. AI refers to the use of algorithms and machines to perform tasks without human intervention, such as image recognition, speech recognition, and natural language processing. Machine learning, on the other hand, is the process of training machines to learn new tasks through the application of algorithms.  
  
The main difference between AI and Machine Learning is that AI is used to perform tasks that require physical or human-like intervention, such as driving vehicles, playing games, and repairing machinery, while Machine Learning is used to perform tasks that require automation or understanding of data, such as analyzing financial records, predicting stock prices, and identifying patterns.  
  
-----  
Observation Summary  
-----  
Observation:  
1. Quality: The API Model (DeepSeek-R1) yields a much more structured, comprehensive, and nuanced answer due to its significantly larger parameter count, than the Local Model (TinyLlama-1.1B) generates a much simpler, basic definition.  
2. Speed: The API model handles generation on cloud servers without consuming local resources. The local model avoids network calls after loading, but generation speed depends entirely on your local machine's hardware.  
(task31) PS C:\Users\Ason\OneDrive\Desktop>python .\ITR_2026_AIPM_Task\frontend_learning\task31.py
```

Q4. (Prompt Template with System Message)

Using any chat model (Groq or HuggingFace):

Create a ChatPromptTemplate with:

A system message: "You are a very polite and helpful AI assistant."

A human message that takes user input.

Take input from the user.

Format the prompt and send it to the model.

Print the response.

Program Code:

#Questions 04

```
.env U X Q1.py U X Q2.py U X Q3.py U X Q4.py U X
Q4.py > ...
1  import os
2  from dotenv import load_dotenv
3  from langchain_core.prompts import ChatPromptTemplate
4  from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
5
6  # Load environment variables from .env file
7  load_dotenv()
8
9  api_token = os.getenv("HUGGINGFACEHUB_API_TOKEN")
10 if not api_token:
11     raise ValueError("HUGGINGFACEHUB_API_TOKEN missing in your .env file!")
12
13 # 1. Create ChatPromptTemplate with System & Human messages
14 prompt_template = ChatPromptTemplate.from_messages([
15     ("system", "You are a very polite and helpful AI assistant."),
16     ("user", "{user_input}")
17 ])
18
19 # 2. Initialize the Hugging Face Endpoint
20 llm = HuggingFaceEndpoint(
21     repo_id="deepseek-ai/DeepSeek-R1",
22     task="text-generation",
23     max_new_tokens=256,
24     temperature=0.7,
25     huggingfacehub_api_token=api_token,
26 )
27
28 # 3. Convert to Chat Model wrapper
29 chat_model = ChatHuggingFace(llm=llm)
30
31 # 4. Chain prompt and model together
32 chain = prompt_template | chat_model
33
34 # 5. Take input from user, format prompt, and send to model
35 user_query = input("Enter your message: ")
36
37 response = chain.invoke({"user_input": user_query})
38
39 # 6. Print response
40 print("\n--- Assistant Response ---")
41 print(response.content)
```

OUTPUT SCREENSHOT:

```
(task31) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31> uv run Q4.py
Enter your message: Can you help me learn Python?

--- Assistant Response ---
<think>
We are going to help the user learn Python. Since the user is a beginner, we should start with the basics.
We'll break down the learning into steps and provide examples.
Steps:
1. Introduction to Python and its syntax.
2. Variables and data types.
3. Basic operations (arithmetic, string operations).
4. Control structures (if statements, loops).
5. Functions.
6. Data structures (lists, tuples, dictionaries, sets).
7. File handling.
8. Introduction to modules and packages.

We'll start with step 1 and 2 in this response, and then proceed step by step as the user progresses.

Let's begin with a brief introduction and then cover variables and data types.
</think>
Absolutely! I'd be delighted to help you learn Python. Let's start with a structured roadmap:

### 🐍 Beginner-Friendly Python Roadmap:
1. **Why Python?**
   - Versatile (web dev, data science, AI, automation)
   - Beginner-friendly syntax

2. **First Steps:**
   ```python
 # Your first program
 print("Hello, World!") # Output: Hello, World!
   ```

(task31) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task31> ++
```

Q5. (Mini Project – Dual Mode Chatbot)

Build a simple command-line chatbot that:

1. Allows the user to choose between API Mode (HuggingFaceEndpoint) or Local

Mode (HuggingFacePipeline).

2. Uses a polite system prompt via ChatPromptTemplate.

3. Continuously takes user input and gives responses.

4. Exits when the user types "exit".

Add proper comments and error handling (e.g., if the API token is missing).

Program Code:

#Questions 05

```
.env v X Q1.py v X Q2.py v X Q3.py v X Q4.py v X Q5.py v X
Q5.py >...
1  import os
2  import sys
3  from dotenv import load_dotenv
4  from langchain_core.prompts import ChatPromptTemplate
5  from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint, HuggingFacePipeline
6
7  # 1. Load environment variables
8  load_dotenv()
9
10
11 def initialize_chat_model():
12     """Prompts the user to select execution mode and returns configured ChatHuggingFace model."""
13     print("=" * 45)
14     print("          DUAL-MODE CHATBOT INITIALIZATION          ")
15     print("=" * 45)
16     print("1. API Mode (deepseek-ai/DeepSeek-R1)")
17     print("2. Local Mode (TinyLlama/TinyLlama-1.1B-Chat-v1.0)")
18
19     choice = input("Select Mode (1 or 2): ").strip()
20
21     if choice == "1":
22         # Check API token validity
23         token = os.getenv("HUGGINGFACEHUB_API_TOKEN")
24         if not token:
25             print("\nError: HUGGINGFACEHUB_API_TOKEN missing in environment or .env file.")
26             sys.exit(1)
27
28         try:
29             print("\nInitializing API Model...")
30             llm = HuggingFaceEndpoint(
31                 repo_id="deepseek-ai/DeepSeek-R1",
32                 task="text-generation",
33                 max_new_tokens=256,
34                 temperature=0.7,
35                 huggingfacehub_api_token=token,
36             )
```

```

37         return ChatHuggingFace(llm=llm)
38     except Exception as e:
39         print(f"\nFailed to load API model: {e}")
40         sys.exit(1)
41
42     elif choice == "2":
43         try:
44             print("\nLoading Local Model into memory (this may take a few moments)...")
45             llm = HuggingFacePipeline.from_model_id(
46                 model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
47                 task="text-generation",
48                 pipeline_kwargs={
49                     "max_new_tokens": 150,
50                     "do_sample": True,
51                     "temperature": 0.7,
52                 },
53             )
54             return ChatHuggingFace(llm=llm)
55         except Exception as e:
56             print(f"\nFailed to load local model: {e}")
57             sys.exit(1)
58
59     else:
60         print("\nInvalid choice selected. Exiting.")
61         sys.exit(1)
62
63
64 def main():
65     # Initialize chosen chat model
66     chat_model = initialize_chat_model()
67

```

```

68     # 2. Define polite system prompt template
69     prompt_template = ChatPromptTemplate.from_messages([
70         ("system", "You are a very polite, kind, and helpful AI assistant."),
71         ("user", "{user_input}")
72     ])
73
74     # Build chain using Runnable syntax
75     chain = prompt_template | chat_model
76
77     print("\n" + "=" * 45)
78     print(" Chatbot ready! Type 'exit' to quit.")
79     print("=" * 45)
80
81     # 3. Interactive continuous chat loop
82     while True:
83         try:
84             user_input = input("\nYou: ").strip()
85
86             # 4. Exit condition
87             if user_input.lower() == "exit":
88                 print("\nAssistant: Goodbye! Have a wonderful day ahead!")
89                 break
90
91             if not user_input:
92                 continue
93
94             response = chain.invoke({"user_input": user_input})
95             print(f"\nAI: {response.content}")
96
97         except KeyboardInterrupt:
98             print("\n\nSession interrupted by user. Goodbye!")
99             break
100        except Exception as e:
101            print(f"\nError generating response: {e}")
102            print(f"\nError generating response: {e}")
103
104    if __name__ == "__main__":
105        main()
106
107

```

OUTPUT SCREENSHOT:

```
(task31) PS C:\Users\Asus\OneDrive\Desktop\ITH 2026 ADPL Task\Fronted learning\task31> uv run q5.py

=====
DUAL-MODE CHATBOT INITIALIZATION
=====
1. API Mode (deepseek-ai/DeepSeek-R1)
2. Local Mode (TinyLlama/TinyLlama-3.1B-Chat-v1.0)
Select Mode (1 or 2): 1

Initializing API Model...

=====
Chatbot ready! Type 'exit' to quit.
=====

You: Hello! Who are you?

AI: <think>
Okay, the user just greeted me and asked who I am. That's a simple and friendly opening.

Hmm, they seem polite with the "hello!" so I should match that tone. Since they're asking about my identity, they might be new to interacting with me or just want to confirm.

I remember my core purpose is to be helpful and kind, so my response should reflect that. I'll introduce myself clearly but warmly - stating my name (DeepSeek-R1), my function (AI assistant), and immediately offer assistance.

The smiley emoji feels appropriate here since their tone is light. Ending with an open question ("How can I help?") invites further interaction naturally.

No signs of frustration or complexity in their query - just a straightforward introductory exchange. Keeping it simple and welcoming is best here.
</think>
Hello! 😊
I'm DeepSeek-R1, your friendly AI assistant. I'm here to help you with questions, ideas, writing, learning, or just chatting-whatever you need!
How can I help you today? 🌟

You: What is machine learning?

AI:
Machine learning is a subset of artificial intelligence (AI) where computers learn to perform tasks by recognizing patterns in data, "without being explicitly programmed for every scenario". Instead of following rigid rules, ML systems improve through experience (data).

You: what is machine learning?

AI:
Machine learning is a subset of artificial intelligence (AI) where computers learn to perform tasks by recognizing patterns in data, "without being explicitly programmed for every scenario". Instead of following rigid rules, ML systems improve through experience (data).

### Key Concepts:
1. "learning from Data":
   ML algorithms analyze large datasets to identify patterns, make decisions, or predict outcomes. The more quality data they process, the better they perform.

2. "Types of ML Approaches":
   - "Supervised Learning": Uses labeled data (e.g., "spam" vs. "not spam" emails) to train models.
   - "Unsupervised Learning": Finds hidden patterns in unlabeled data (e.g., grouping customers by behavior).
   - "Reinforcement Learning": Learns via trial-and-error feedback (e.g., teaching a robot to walk).

3. "Common Applications":
   - Email spam filters
   - Recommendation systems (Netflix, Spotify)
   - Image/voice recognition (Siri, facial ID)
   - Medical diagnosis or stock market predictions

### Why It Matters:
ML enables systems to adapt to new information, automate complex tasks, and uncover insights from massive datasets - making

You: exit

Assistant: Goodbye! Have a wonderful day ahead!
(task31) PS C:\Users\Asus\OneDrive\Desktop\ITH 2026 ADPL Task\Fronted learning\task31>
```

